



Politehnica University of Timisoara



# Web Programming

## C5. Rich Internet Application (RIA)



# RIA

---

- **Rich Internet Application (RIA)** - a web application that behave close to a desktop application
- Main issues:
  - Interactivity / performance / scalability
  - Responsiveness / user interface
  - Browsers / platform compatibility



# RIA Technologies

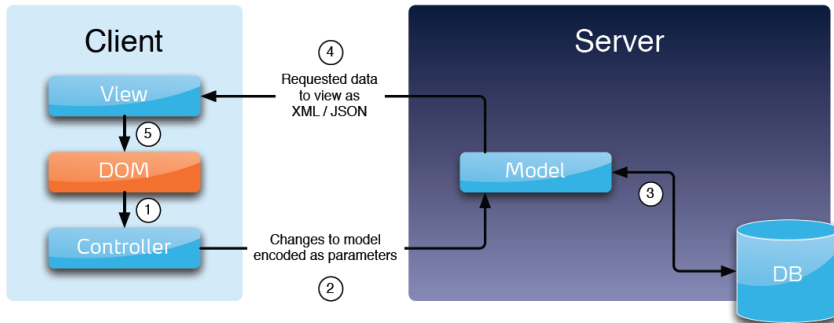
- Common technologies for Web apps:
  - SSS (PHP, Python etc.)+ CSS (JS)+ Ajax
  - Various Ajax-based Frameworks
  - Jakarta EE (formerly Java EE) 
  - Java + Java-to-Javascript (e.g., Vaadin) 
  - Ruby + Ruby on Rails 
  - Adobe Flash + Air 
  - Microsoft Silverlight / (free, Unix) Moonlight  



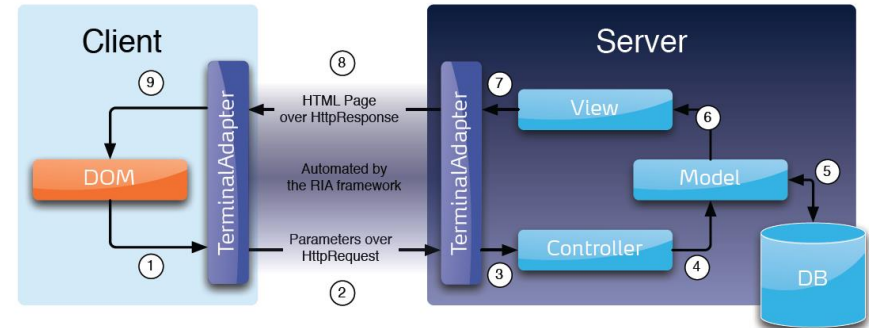
# Two RIA Design Philosophies



## Fat Web Client



## Thin Web Client



\* MVC solutions (many alternatives: MVVM, MVP, MVI)



# Model View Controller

---

- Model – data model and the business logic
- View – for presenting the user interface
- Controller - mediator between the Model and the View
- Advantages
  - Maintainability: due separation of concerns (data, presentation, and control)
  - Testability: components can be tested independently
  - Reusability: components can be reused (e.g., MVC frameworks)
  - Clear Data Flow: trace how input triggers changes in the Model and View
- Limitations
  - Tight Coupling Between Components results in heavy controller code, and can make it harder to isolate and test specific behaviors
  - Difficulty in Managing Large-Scale Applications with complex data-flows
  - Lack of Flexibility to Changing Requirements – additional code to keep the View and Model synchronized



# MVVM (Model View ViewModel)

---

- ViewModel - retrieves data from the Model, prepares it for presentation, and exposes it to the View through *data binding* mechanisms
- Advantages
  - Two-Way Data Binding: changes in the View are automatically reflected in the Model (and vice versa), simplifying data synchronization and user interaction handling
  - Improved Maintainability: the ViewModel encapsulates data presentation logic, leading to cleaner and more maintainable Views
  - Platform Independence: the ViewModel can be platform-independent, while the View implementation can be tailored to specific platforms
- Represents a good choice for web app development frameworks targeting mobile platforms and for projects where frequent UI updates are necessary



# MVP (Model View Presenter)

---

- Presenter – fetches data from the Model and prepares it for the View in a suitable format
- View has no direct knowledge of the Model
- Advantages
  - Improved Testability: for both the View (focusing on UI behavior) and the Presenter (handling business logic)
  - Improved maintainability: due to loose coupling, changes in the View do not directly impact the Presenter or the Model, and vice versa
  - Focus on UI: the View can solely focus on presentation, making it easier to build reusable UI components
- Represents a good choice for projects that
  - Implement complex user interfaces
  - Have critical testability and maintainability requirements



# MVI (Model View Intent)

---

- MVI implements unidirectional data flow
- View dispatches user intents (actions) to a central store
- Store holds the application state and updates the Model based on received intents, then sends a new state back to the View
- Advantages
  - Predictable State Management: unidirectional data flow simplifies reasoning about application behavior and makes debugging easier
  - Testability: excellent testability for each component (View, Intent, and Model)
  - Complexity: a steeper learning curve due to its emphasis on functional programming concepts and unidirectional data flow
- Represents a good choice for reactive applications where state management and immutability are key concerns



# Laminas Framework (Zend)

La///inas

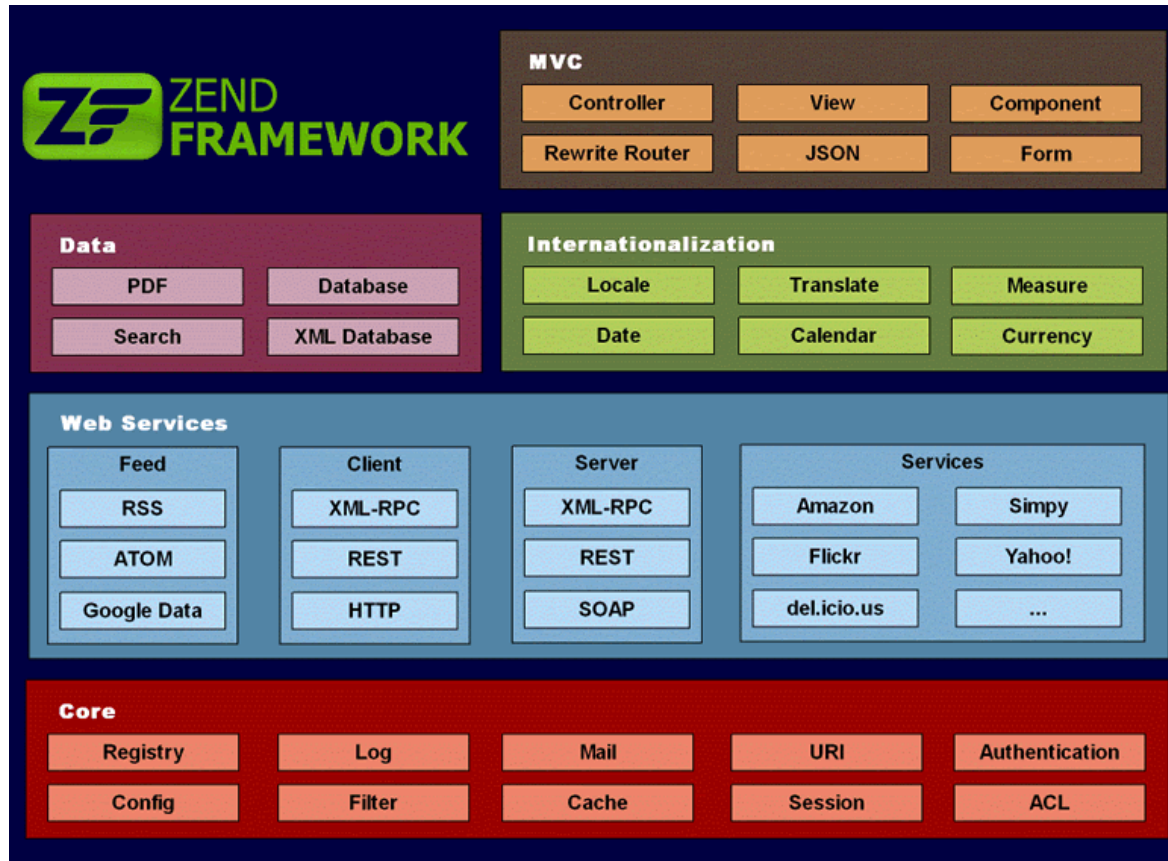
<https://getlaminas.org/>



- A PHP/Ajax framework for complex RIA applications
- Both Thin/Fat Web Client based on Ajax
- OOP code in PHP on the server side
- Large collection of components for Enterprise Applications + UI



# The framework architecture



Ref: [williamxjx.wordpress.com/tag/zend-framework/](http://williamxjx.wordpress.com/tag/zend-framework/)



# Laminas Templating System

```
1 use Laminas\View\Model\ViewModel;
2
3 // Create the layout view model:
4 $layout = new ViewModel([
5     'encoding' => 'utf-8',
6     'cssPath'  => '/css/blog/',
7 ]);
8 $layout->setTemplate('layout::layout');
9
10 $content = $renderer->render('blog/entry', [
11     'layout' => $layout,
12     'entry'  => $entry,
13 ]);
```

Code Ref: <https://docs.mezzio.dev/mezzio/v3/features/template/laminas-view/>



# Laminas

---

## ■ Advantages:

- Modular and customizable components
- Extensive database support
- Reusability of components
- Strong focus on security
- Comprehensive documentation
- Excellent AJAX support

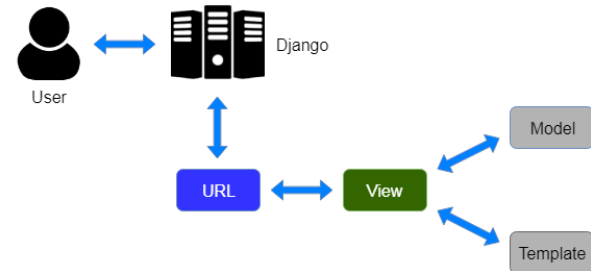
## ■ Limitations

- Increased complexity for smaller projects
- Smaller community compared to other frameworks
- Effort required for maintaining complex coding practices



# Django Framework

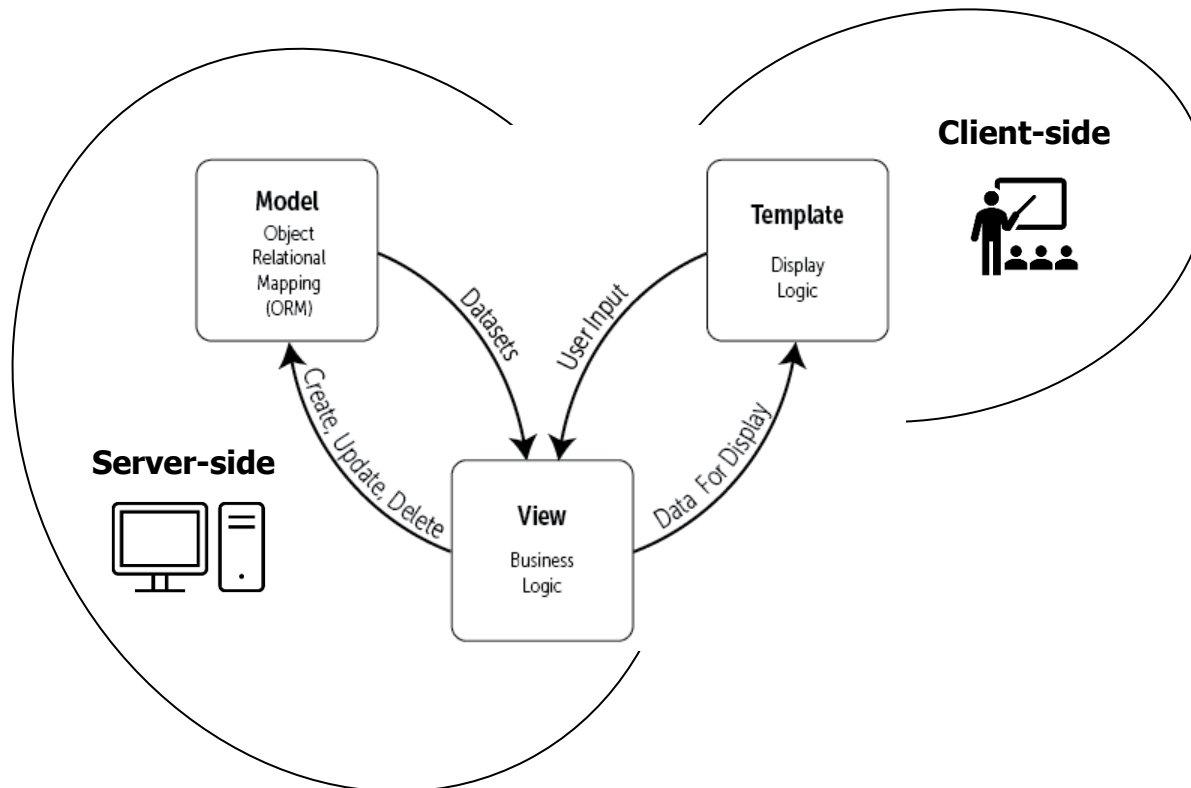
- A Python open-source Web framework
- Based on the MVT architectural pattern
- Primary goal:
  - Simplifying the creation of complex database-driven RIA as thin web client
  - Supporting reusability of components
  - Facilitating rapid app development





# Django MVT

- Model-View-Template (MVT) architecture





# Django Architecture

## ■ Models

- Data access layer
- Each model maps to a single DB table

```
from django.db import models

class Person(models.Model):
    first_name = models.CharField(max_length=30)
    last_name = models.CharField(max_length=30)
```

```
CREATE TABLE myapp_person (
    "id" serial NOT NULL PRIMARY KEY,
    "first_name" varchar(30) NOT NULL,
    "last_name" varchar(30) NOT NULL
);
```

## ■ Views

- Python functions
- Each view takes a request and returns a HTTP response (HTML or XML)

```
from django.http import HttpResponse
import datetime

def current_datetime(request):
    now = datetime.datetime.now()
    html = "<html><body>It is now %s.</body></html>"
    return HttpResponse(html)
```



# Django Templates

## ■ Template

- Use `{{x}}` as placeholder for variable `x`

`//index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Index</title>
</head>
<body>
<h2>Welcome to Django!!!</h2>
<h3>My Name is: {{ student }}</h3>
</body>
</html>
```

## ■ Views

- Render the template

`//views.py`

```
from django.shortcuts import render
# importing loading from django template
from django.template import loader
# Create your views here.
from django.http import HttpResponse
def index(request):
    template = loader.get_template('index.html')
    name = {
        'student': 'rahul'
    }
    return HttpResponse(template.render(name))
```



# Vaadin

---

**vaadin** }> <https://demo.vaadin.com/sampler/>

- Vaadin – a Java framework for RIA applications
- Thin Web Client based on Google Web Toolkit (GWT - [www.gwtproject.org/](http://www.gwtproject.org/))
- Pure Java code on the server side
- Large collection of UI components





# Vaadin UI

```
@Route("hello")
public class HelloWorldView extends VerticalLayout {

    public HelloWorldView(@Autowired GreetService service) {
        TextField name = new TextField("Name");
        Paragraph greeting = new Paragraph("");

        Button sayHello = new Button("Greet");
        sayHello.addThemeVariants(ButtonVariant.LUMO_PRIMARY);
        sayHello.addClickListener(e -> {
            greeting.setText(service.greet(name.getValue()));
        });

        add(name, sayHello, greeting);
    }
}
```

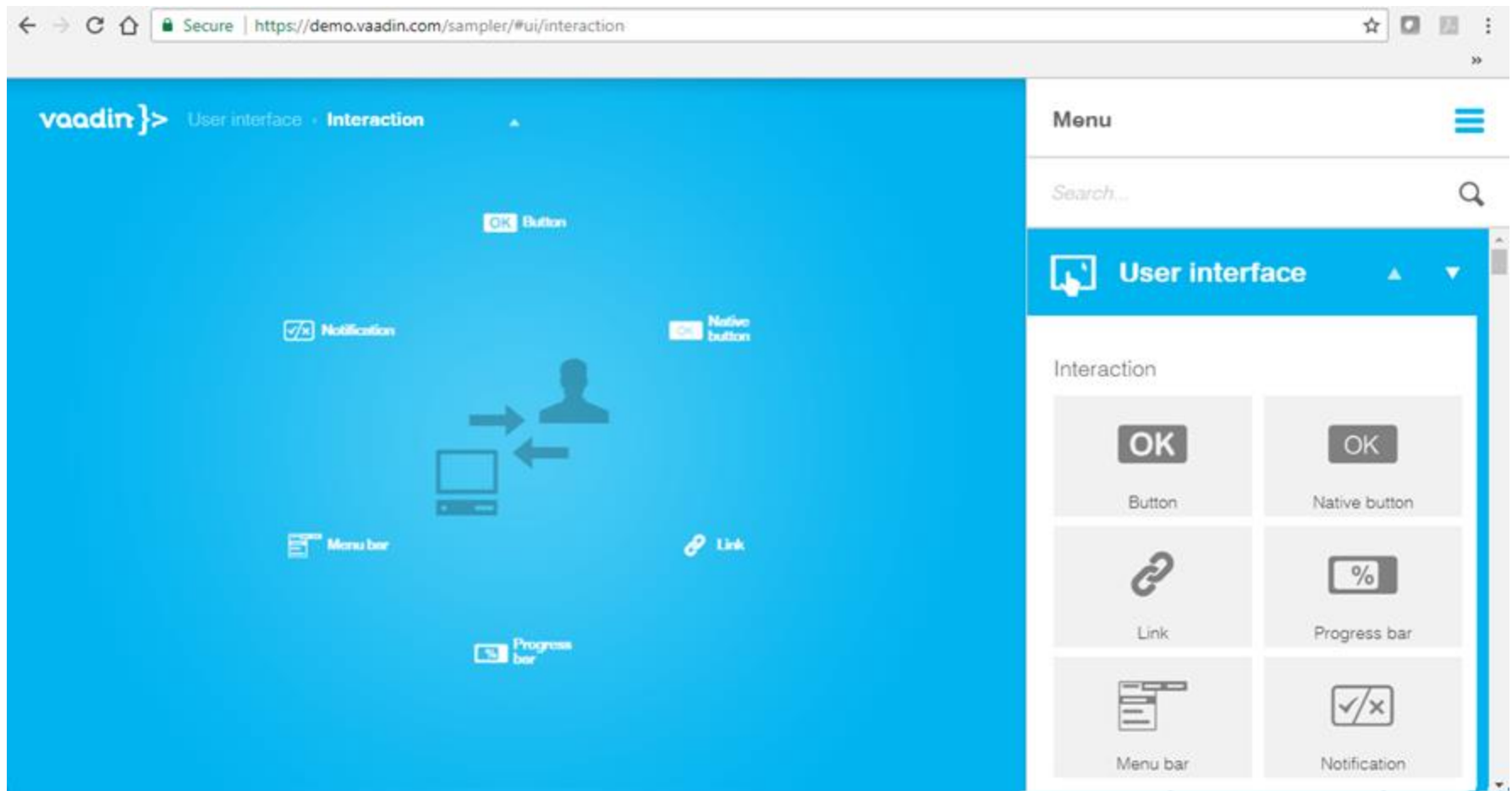
Name

Greet

Hello Vasile!

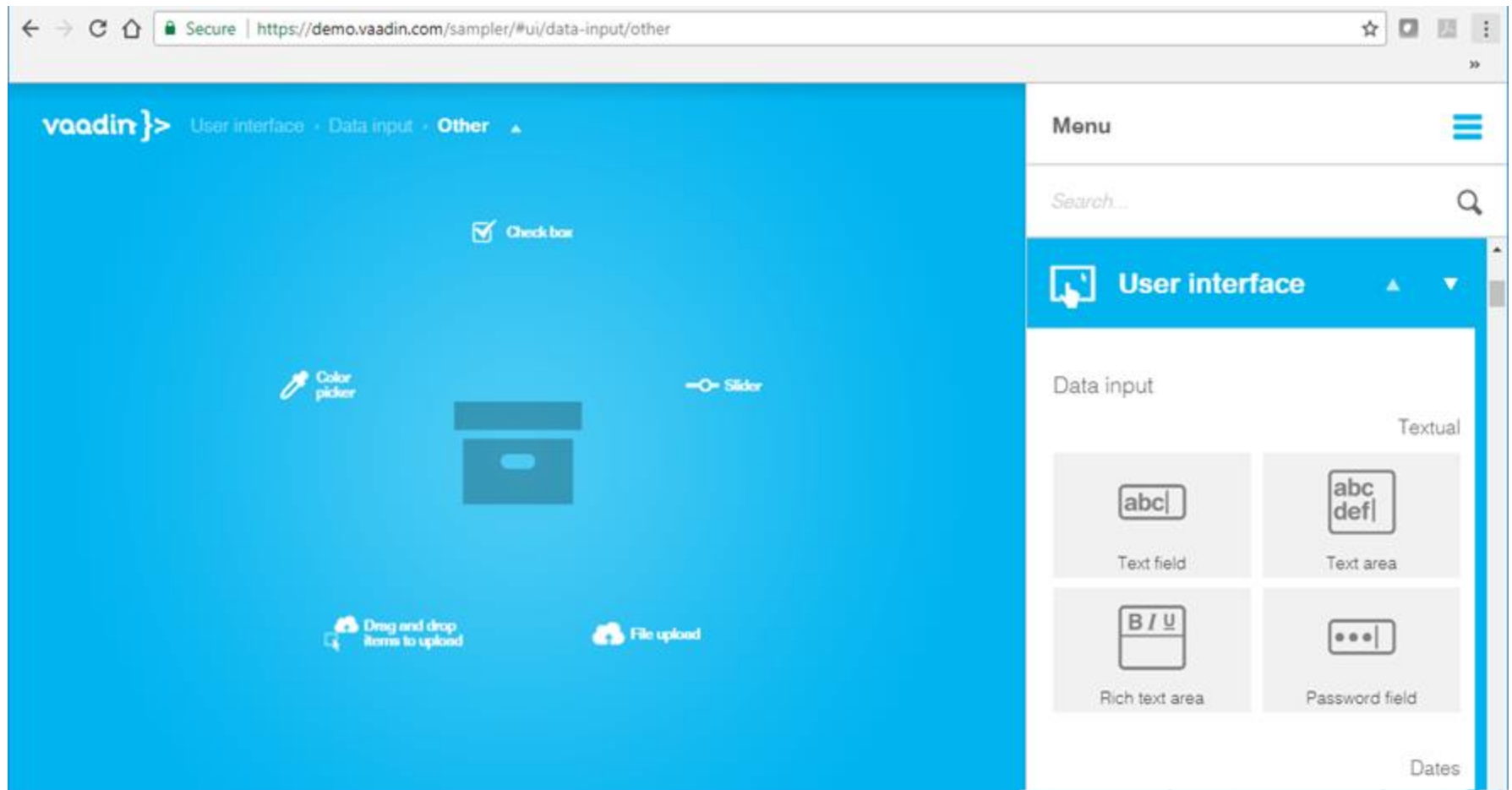


# Vaadin



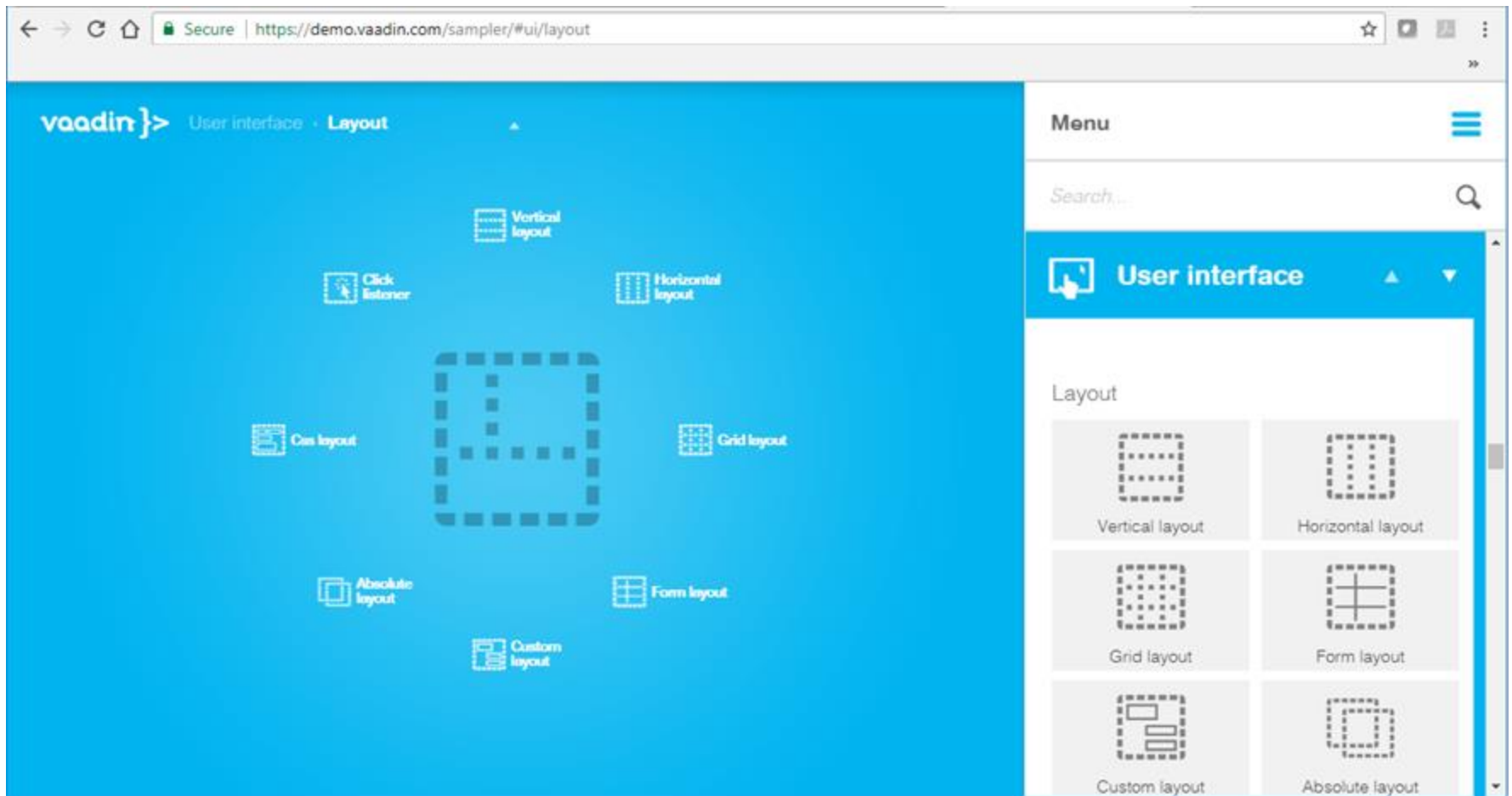


# Vaadin





# Vaadin





Secure | <https://demo.vaadin.com/sampler/#ui/data-input/dates/date-picker>

Comp150WEB -- Adv. | A Practical Introduction | Rackspace Webmail | Concedii 2017 | Tech Support Shifts | Dan Pescaru | Publon | Sign in to your account | Planning - OneDrive | [GENRES-35] Holding

**vaadin** > User interface > Data input > Dates > **Date picker**

Information   

**Description** **Source**

1. `sample = new InlineDateTimeField();`  
2. `sample.setValue(LocalDate.now());`  
3. `sample.setLocale(Locale.US);`  
4. `sample.setResolution(DateTimeResolution.MINUTE);`  
5.  
6. ...  
7.  
8. `sample.addValueChangeListener(event -> Notification.show("Value changed:",`  
9. `String.valueOf(event.getValue()),`  
10. `Type.TRAY_NOTIFICATION));`

« < February 2018 > »

| Sun | Mon | Tue | Wed | Thu | Fri | Sat |
|-----|-----|-----|-----|-----|-----|-----|
| 28  | 29  | 30  | 31  | 1   | 2   | 3   |
| 4   | 5   | 6   | 7   | 8   | 9   | 10  |
| 11  | 12  | 13  | 14  | 15  | 16  | 17  |
| 18  | 19  | 20  | 21  | 22  | 23  | 24  |
| 25  | 26  | 27  | 28  | 1   | 2   | 3   |
| 4   | 5   | 6   | 7   | 8   | 9   | 10  |

12 : 44 PM



# Google Web Technology

---

- Useful widgets for web page content enhancement
    - **Maps**
    - **Wave**
    - **Wallet**
    - **Firestore NoSQL DB**
    - **Flutter / Dart Framework**
    - **Generative AI (Gemini API)**
- <https://developers.google.com/>



# Google API

Google APIs Explorer

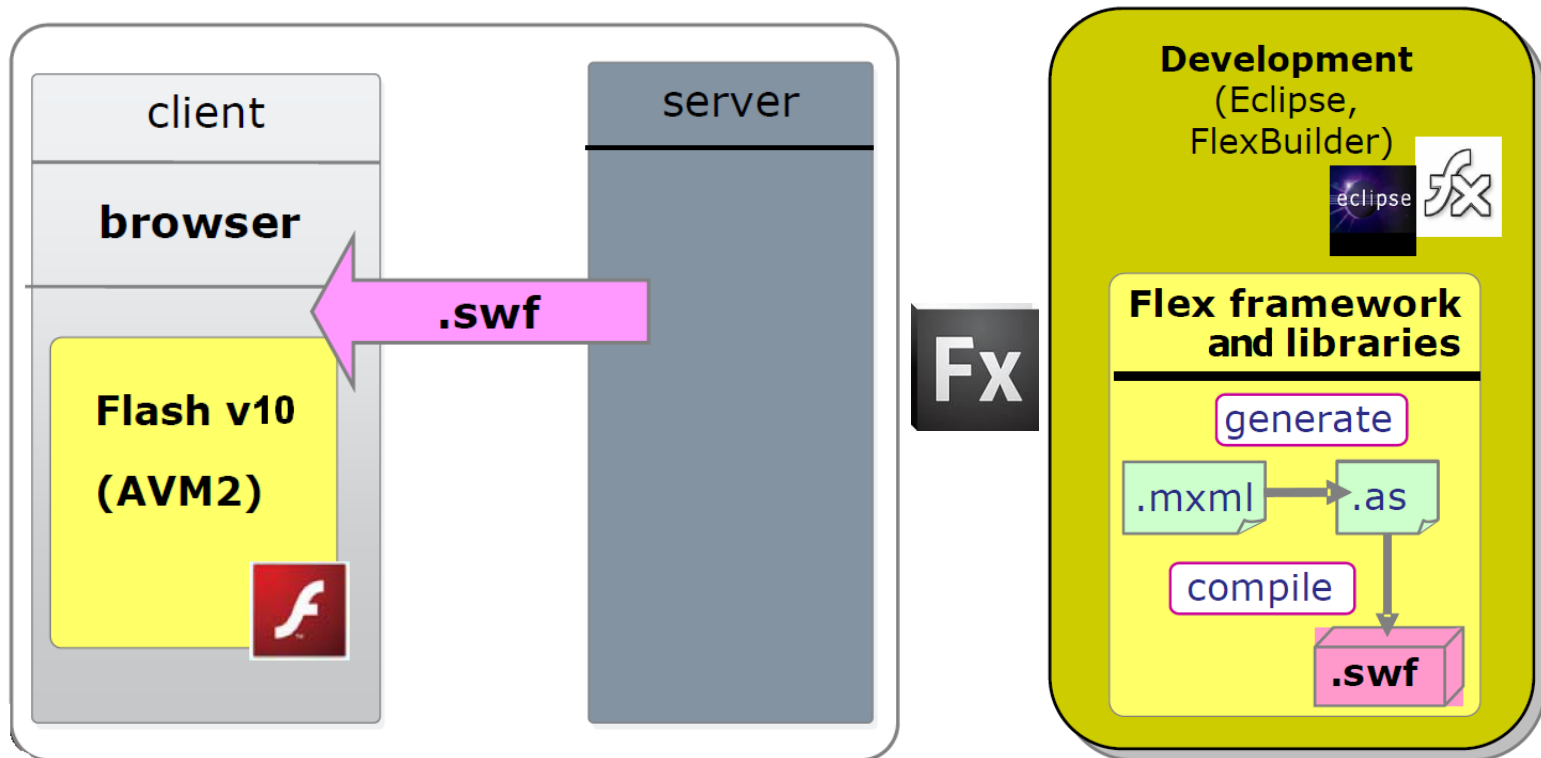
Google APIs Explorer 

Type to filter APIs

| Title                                                     | Description                                                                                                                                                     |
|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">Abusive Experience Report API</a>             | Views Abusive Experience Report data, and gets a list of sites that have a significant number of abusive experiences.                                           |
| <a href="#">Accelerated Mobile Pages (AMP) URL API</a>    | Retrieves the list of AMP URLs (and equivalent AMP Cache URLs) for a given list of public URL(s).                                                               |
| <a href="#">Access Context Manager API</a>                | An API for setting attribute based access control to requests to GCP services.                                                                                  |
| <a href="#">Ad Exchange Buyer API II</a>                  | Accesses the latest features for managing Authorized Buyers accounts, Real-Time Bidding configurations and auction metrics, and Marketplace programmatic deals. |
| <a href="#">Ad Experience Report API</a>                  | Views Ad Experience Report data, and gets a list of sites that have a significant number of annoying ads.                                                       |
| <a href="#">adexchangebuyer v1.2</a>                      |                                                                                                                                                                 |
| <a href="#">adexchangebuyer v1.3</a>                      |                                                                                                                                                                 |
| <a href="#">adexchangebuyer v1.4</a>                      |                                                                                                                                                                 |
| <a href="#">Admin SDK API 1</a>                           | Admin SDK lets administrators of enterprise domains to view and manage resources like user, groups etc. It also provides audit and usage reports of domain.     |
| <a href="#">Admin SDK API 1</a>                           | Admin SDK lets administrators of enterprise domains to view and manage resources like user, groups etc. It also provides audit and usage reports of domain.     |
| <a href="#">Admin SDK API 1</a>                           | Admin SDK lets administrators of enterprise domains to view and manage resources like user, groups etc. It also provides audit and usage reports of domain.     |
| <a href="#">AdMob API</a>                                 | The AdMob API allows publishers to programmatically get information about their AdMob account.                                                                  |
| <a href="#">AdSense Host API</a>                          | The AdSense Host API gives AdSense Hosts access to report generation, ad code generation, and publisher management capabilities.                                |
| <a href="#">AdSense Management API v1.4</a>               | The AdSense Management API allows publishers to access their inventory and run earnings and performance reports.                                                |
| <a href="#">AdSense Management API v2</a>                 | The AdSense Management API allows publishers to access their inventory and run earnings and performance reports.                                                |
| <a href="#">AI Platform Training &amp; Prediction API</a> | An API to enable creating and using machine learning models.                                                                                                    |
| <a href="#">Analytics Core Reporting API v3</a>           | Query for dimensions and metrics to produce customized reports.                                                                                                 |
| <a href="#">Google Analytics Data API</a>                 | Accesses report data in Google Analytics.                                                                                                                       |



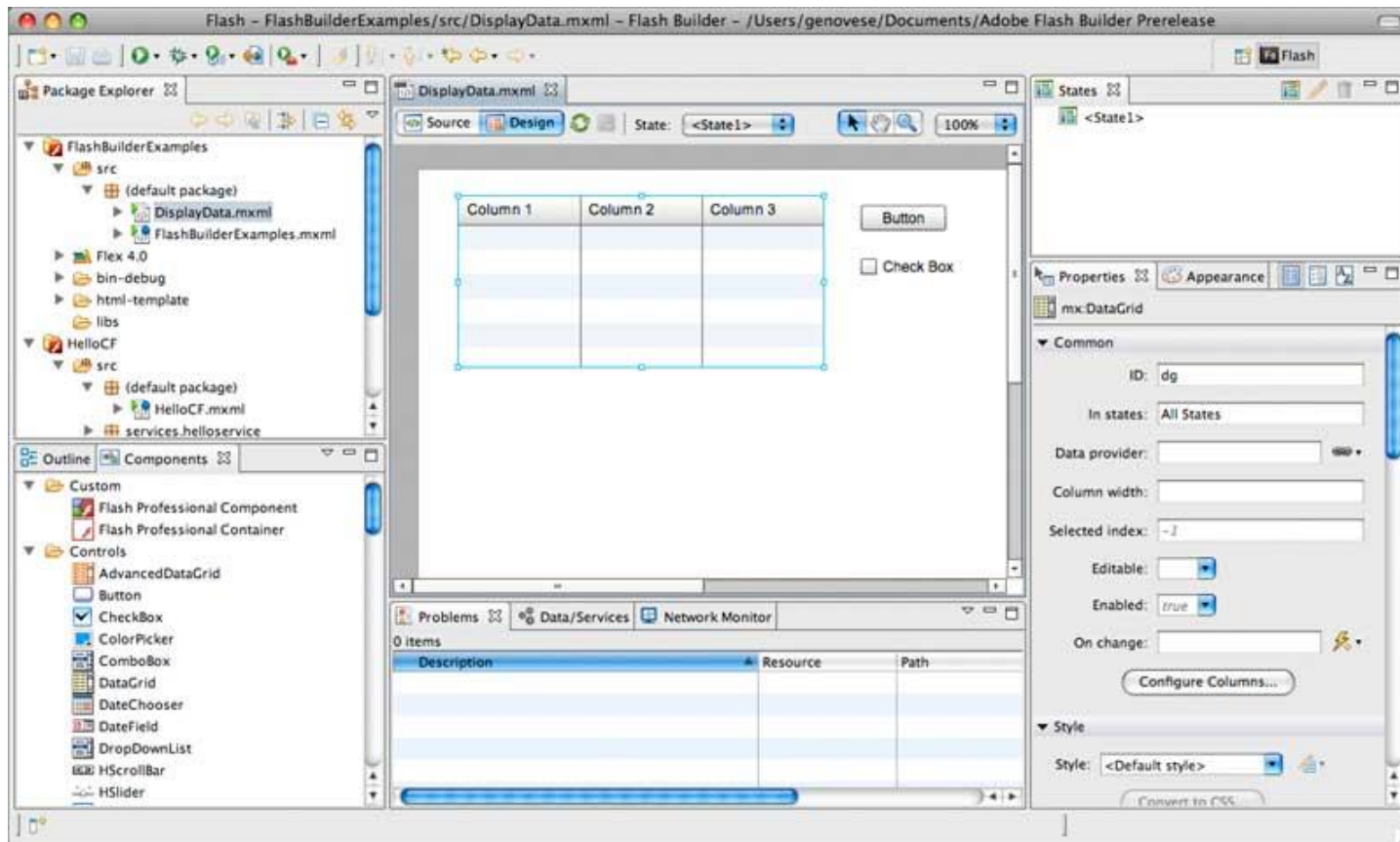
# Adobe Flex, MXML and AS



<http://www.adobe.com/devnet/flex.html>



# Adobe Flash Builder





# Apache Royale

---

- An open-source project
- Development of applications close to Flex philosophy
  - ActionScript 3
  - MXML for UI
- Generate JavaScript (no need for Flash Player)





# Apache Royale resources

Apache Royale Tour – Web application for exploring ActionScript & MXML components (<https://royale.apache.org/tourdejewel/>)

Apache Royale Tour de Jewel

EXAMPLES SOURCE CODE

Watch the source code for this page in GitHub

Jewel Table

Jewel Table examples.

Jewel SimpleTable

| Icons | Text Column                 | Centered Text Column | Number Column |
|-------|-----------------------------|----------------------|---------------|
|       | Some text in two rows       | Some centered text   | 71            |
|       |                             | More centered text   | 56            |
|       | More text 2                 | More centered text 2 | 346           |
|       | Labels with Vertical Layout | More centered text 3 | 240           |

Search... SEARCH

```
<?xml version="1.0" encoding="utf-8"?>
<!--
Licensed to the Apache Software Foundation (ASF) under one or more
contributor license agreements. See the NOTICE file distributed with
this work for additional information regarding copyright ownership.
The ASF licenses this file to You under the Apache License, Version 2.0
(the "License"); you may not use this file except in compliance with
the License. You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
-->
<:ExampleAndSourceCodeTabbedSectionContent xmlns:fx="http://ns.adobe.com/mxml/2009"
xmlns:j="library://ns.apache.org/royale/jewel"
xmlns:html="library://ns.apache.org/royale/html"
xmlns:js="library://ns.apache.org/royale/basic"
xmlns:c="components.*"
xmlns:models="models.*" sourceCodeUrl="TablePlayGround.mxml">

<c:model>
<models:TablesModel localId="tablesModel"/>
</c:model>

<c:beads>
<js:ContainerDataBinding/>
</c:beads>
```



# Web Services Architecture

- W3C: a software system designed to support interoperable machine-to-machine interaction over a network
- Interface - Web Services Description Language (WSDL)
- SOAP messages over HTTP (XML serialization)





# Web Services

---

- Are application components
- Communicate using open protocols
- Are self-contained and self-describing
- Can be discovered using Universal Description, Discovery and Integration (UDDI)
- Use XML for data exchange



# Web Services C# example

```
FirstService.asmx
<%@ WebService language = "C#" class = "FirstService" %>

using System;
using System.Web.Services;
using System.Xml.Serialization;

[WebService(Namespace = "http://localhost/MyWebServices/")]
public class FirstService : WebService{
    [WebMethod]
    public int Add(int a, int b) {
        return a + b;
    }

    [WebMethod]
    public String SayHello() {
        return "Hello World";
    }
}
```

Ref: <https://www.tutorialspoint.com/webservices>



# WSDL

---

- Web Services Description Language
- Is a W3C recommendation
- Based on XML
- Used to describe Web services and the way to access them
- Used to locate Web services



# WSDL Documents

---

- WSDL document structure:

## **<definitions>**

<types>

// data type definitions

</types>

<message>

// definition of the data being communicated and associated data types

</message>

<portType>

// set of operations that can be performed, inputs and outputs names

</portType>

<binding>

// protocol and data format specification, usually SOAP

</binding>

**</definitions>**



# SOAP

---

- Simple Object Access Protocol
- Designed to communicate via Internet
- Platform independent
- Language independent
- Based on XML
- Simple and extensible
- Pass firewalls – based on HTTP



# SOAP Messages

---

- SOAP message structure:

```
<?xml version="1.0"?>
<soap:Envelope
xmlns:soap="http://www.w3.org/2003/05/soap-envelope/"
soap:encodingStyle="http://www.w3.org/2003/05/soap-encoding">
  <soap:Header>
    - application-specific information
  </soap:Header>
  <soap:Body>
    - actual message data
    <soap:Fault>
      - indicate error messages
    </soap:Fault>
  </soap:Body>
</soap:Envelope>
```



# UDDI

---

- Universal Description, Discovery and Integration registry
- A directory for storing information about web services
- Exposes services interfaces - e.g. described in WSDL
- Communicates via SOAP
- Implementation on Microsoft .NET platform



# Web Mashups

- A web application that takes information from one or more sources and presents it in a new way or with a unique layout (D. Nations)
- Example:  
<http://flickrsudoku.com/>  
uses Flickr number photos as symbols for Sudoku game





# Progressive Web Apps

---

- Web apps that are:
  - **Installable** as a “normal” app on the native platform (e.g., install an icon on the desktop which launches the app)
  - **Progressively enhanced** to adapt the complexity of the app content (features) to the user resources (HW, Browser, OS ...)
  - **Responsively designed** to provide UI that fits any form factor as desktop monitor, mobile, tablet, or whatever comes
  - **Re-engageable** via *Push API* for sending updates straight from the server to the app via Service Workers
  - **Linkable** as the ability to link to an app at a URL without the need for an app store or complex installation process
  - **Network independent** to work when the network is unreliable



# Progressive Web Apps

---

- Progressive web apps technologies:
  - **Service Worker API** - a JavaScript prog that can control the web-page that it is associated with, intercepting and modifying navigation and resource requests, and caching resources in a very granular fashion to give control over how the app behaves in certain situations (e.g., when the network is not available)
  - **Web Storage API** provides mechanisms in browsers to store key/value pairs, in a much more intuitive fashion than using cookies
  - **Cache API** - provides a persistent storage mechanism for Request/Response object pairs that are cached in long lived memory on the browser side



# Windmill – for testing RIA

---

- A web testing tool created to automate the debugging process for web applications
- Possible environments:
  - The “shell” starts a Phyton shell
  - The “run\_service” runs windmill in the foreground until it gets a proper signal to end
  - The “wx” launch an wxPython UI for the service





# Selenium - for testing RIA

- A portable open source Javascript based web testing tool for Web 2.0 apps
- Advantages:
  - Language independent through drivers
  - Monitor DOM for completion of Async calls
  - Implements waitForCondition, waitForValue for Ajax
  - Generates HTML reports
  - Implements Driven/Record/Grid modes





# RIA - resources

---

- Progressive Web Apps

<https://web.dev/progressive-web-apps/>

- Web Services Tutorial (w3schools)

<https://w3schools.sinsixx.com/webservices/default.asp.htm>

- Selenium Browser Automation Project

<https://www.selenium.dev/documentation/>